

Riwaz Shrestha

Mar 7, 2025

DS210

Colaborators: None

Homework 4 Report

Problem 1.

Code:

```
use std::ops::Neg;

fn main() {
    let p1:Point<f32> = Point{x: 1.32, y: 3.33};
    let p2:Point<i64> = Point{x: 1976, y: 223};
    let p1_clock = p1.clockwise();
    let p2_counter = p2.counterclockwise();
    println!("Original P1: {:?} \nClockwise P1: {:?}",p1, p1_clock);
    println!();
    println!("Original P2: {:?} \nCounterclockwise P2: {:?}", p2, p2_counter);
}

#[derive(Debug, Clone, Copy, PartialEq)]
struct Point<T> {
    x:T,
    y:T,
}

impl <T:Copy + Neg<Output = T>> Point<T>{
    fn clockwise(&self) -> Point<T>{
        Point {
            x: self.y,
            y: -self.x,
        }
    }

    fn counterclockwise(&self) -> Point<T>{
        Point { x: -self.y, y: self.x }
    }
}

#[test]
fn clockwise_works(){
```

```

    let clock: Point<f64> = Point{x:1.0, y:3.0};
    let result = clock.clockwise();
    let expected: Point<f64> = Point{x:3.0,y:-1.0};
    assert_eq!(result, expected, "Fail!!!");
}

#[test]
fn counterclockwise_works() {
    let clock: Point<f64> = Point{x:1.0, y:3.0};
    let result = clock.counterclockwise();
    let expected: Point<f64> = Point{x:-3.0,y:1.0};
    assert_eq!(result, expected, "Fail!!!");
}

```

Explanation:

I made a struct called point that can take in any arbitrary value of T. T is a generic; it can take in any type. This code requires T to be a numerical value. The Impl statement takes into account the different traits that will be used. The function clockwise turns the point 90 degrees clockwise while the function counterclockwise turns the point 90 degrees counterclockwise.

The two tests check whether the functions work or not by comparing the rotated points to the calculated points after rotation. They passed.

Output:

Original P1: Point { x: 1.32, y: 3.33 }

Clockwise P1: Point { x: 3.33, y: -1.32 }

Original P2: Point { x: 1976, y: 223 }

Counterclockwise P2: Point { x: -223, y: 1976 }

Explanation:

In the main function, I create two different points and rotate each either 90 degrees clockwise or 90 degrees counterclockwise. Rotating 90 degrees in either direction flips the x and y values. Rotating clockwise makes the y value negative, while rotating counterclockwise makes the x value negative.

Problem 2.

Code:

```
fn main() {
    let size: usize = 24;
    let mut first = Board::create(size);

    first.board[0][2] = 1;
    first.board[1][2] = 1;
    first.board[2][0] = 1;
    first.board[2][1] = 1;
    first.board[2][2] = 1;

    first.print();
    println!();

    for _ in 0..4{

        let mut next = Board::create(size);

        for i in 0..first.board.len(){
            for j in 0..first.board.len(){
                first.next_iteration(&mut next, i, j);
            }
        }

        next.print();
        println!();

        first = next;
    }
}

#[derive(PartialEq)]
struct Board{
    size: usize,
    board: Vec<Vec<usize>>,
}

impl Board{
    fn create(x:usize) -> Self{
        Board{
            size:x,
```

```

        board:vec![vec![0;x];x]
    }
}

fn num_neighbors(&self, x:usize, y:usize) -> usize{
    let diff = [-1,0,1];
    let mut sum:usize = 0;
    for xdiff in diff{
        for ydiff in diff{
            if xdiff == 0 && ydiff == 0{
                continue;
            }
            let nx = (x as isize + xdiff + self.size as isize) % self.size as
isize;

            let ny = (y as isize + ydiff + self.size as isize) % self.size as
isize;

            if nx != x as isize || ny != y as isize{
                if self.board[nx as usize][ny as usize] == 1{
                    sum += 1;
                }
            }
        }
    }
    return sum
}

fn next_iteration(&self, other: &mut Board, x:usize, y:usize){
    let neighbors = self.num_neighbors(x,y);
    match neighbors{
        3 => other.board[x][y] = 1,
        2 => other.board[x][y] = self.board[x][y],
        _ => other.board[x][y] = 0,
    }
}

fn print(&self){
    for i in &self.board{
        for j in i{
            print!("{}", if *j == 1 {"🐱"} else {"■"});
        }
        println!();
    }
}
}

```

```

//Tests
// make a small version of the game using 3x3 grid
// 3x3 grid: all other points will be neighbors
// make your own 3x3 grid and use for asserteq
// source - Zach Gentile

#[test]
fn lonely_pig_dies(){
    let size: usize = 3;
    let mut first = Board::create(size);
    first.board[0][0] = 1;
    for _ in 0..1{
        let mut next = Board::create(size);
        for i in 0..first.board.len(){
            for j in 0..first.board.len(){
                first.next_iteration(&mut next, i, j);
            }
        }
        first = next;
    }
    let correct = Board::create(size);
    assert_eq!(first, correct, "FAIL!!!");
}

#[test]
fn square_of_pigs(){
    let size: usize = 4;
    let mut first = Board::create(size);
    first.board[1][1] = 1;
    first.board[1][2] = 1;
    first.board[2][1] = 1;
    for _ in 0..5{
        let mut next = Board::create(size);
        for i in 0..first.board.len(){
            for j in 0..first.board.len(){
                first.next_iteration(&mut next, i, j);
            }
        }
        next.print();
        println!();
        first = next;
    }
}

```

```

let mut correct = Board::create(size);
correct.board[1][1] = 1;
correct.board[1][2] = 1;
correct.board[2][1] = 1;
correct.board[2][2] = 1;
assert_eq!(first, correct, "FAIL!!!");
}

```

Explanation:

I started by creating a board struct to have an easy-to-create board with implementations that were easy to use. The struct contains the size of the board and the board itself. This struct derives from `PartialEq`. `PartialEq` was for the testing. Debug was unnecessary this time, as I was only printing char values. In the `impl` statement, I had functions to create the board, find the number of neighbors, run the next iteration of the board, and print the board using special characters.

In the `create` function, I created the board using a variable of the size of the board. This function returned a `Board` struct.

The function `num_neighbors` took in itself and a point and counted the number of neighbors around it. It iterated through `[-1,0,1]` for the x and y axes to get its neighbors. The point itself did not count because of the `if` statement making sure that the code ran to the next iteration if the distance from the point was zero. It checked every point around it to see if there was a 1 (alive) or 0 (dead). For every alive point, the sum of neighbors increased by 1. After the program checks all 8 points around the original point, the function returns the total number of neighbors.

The function `next_iteration` used took in itself, another mutable board and a point. This function used the `num_neighbors` function to find the number of neighbors for the given point and then matched the number of neighbors to the correct response on the other board. I use a `match` statement to match the number of neighbors. If there were 3 neighbors, the point became alive. 2 meant that it stayed as is. Any other number of neighbors meant that it died. I guess that any more than 3 neighbors meant either conflict leading to death or overcompetition leading to death. I updated the same point on the new board accordingly.

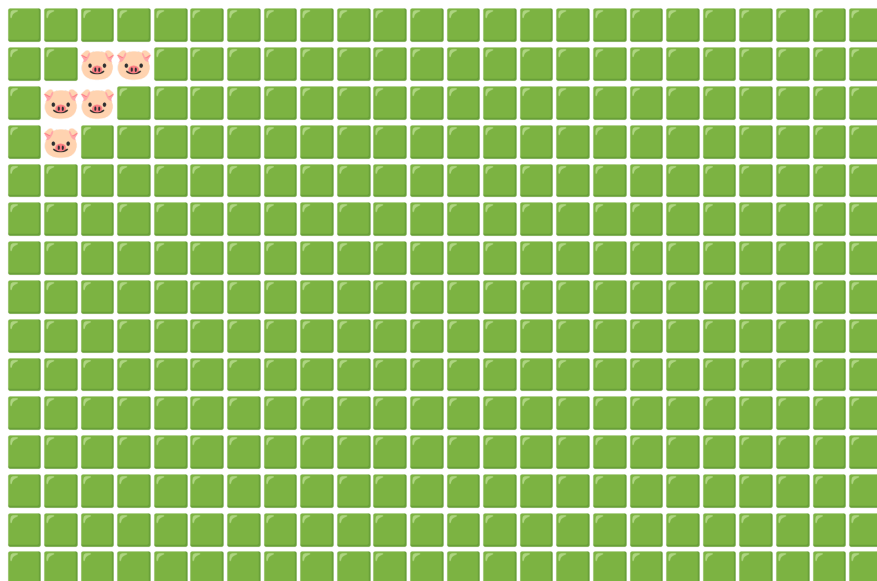
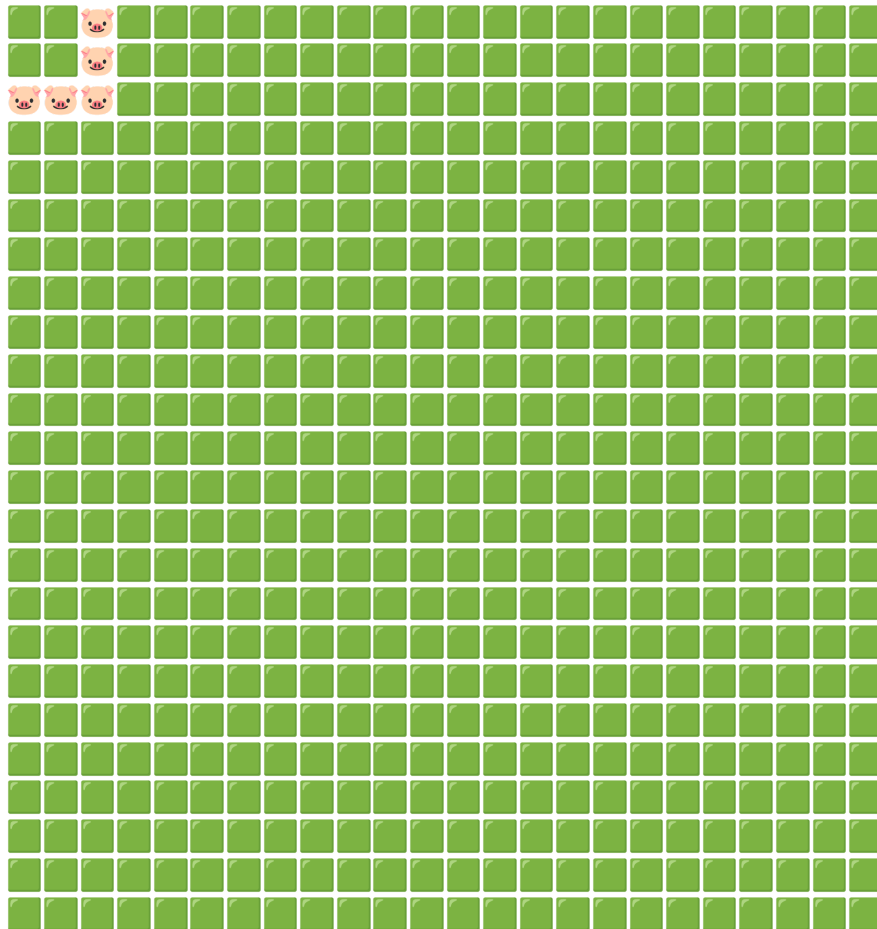
The last function prints out the board using a nested loop running through each point. If the point is alive, it is represented by the pig character, and if the point is dead, it is represented by a green square.

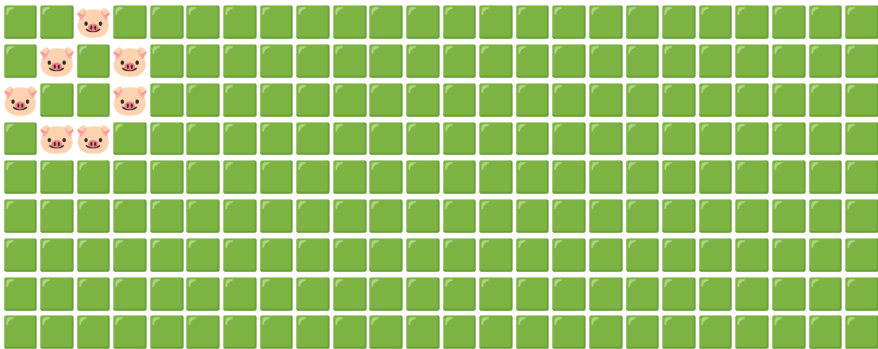
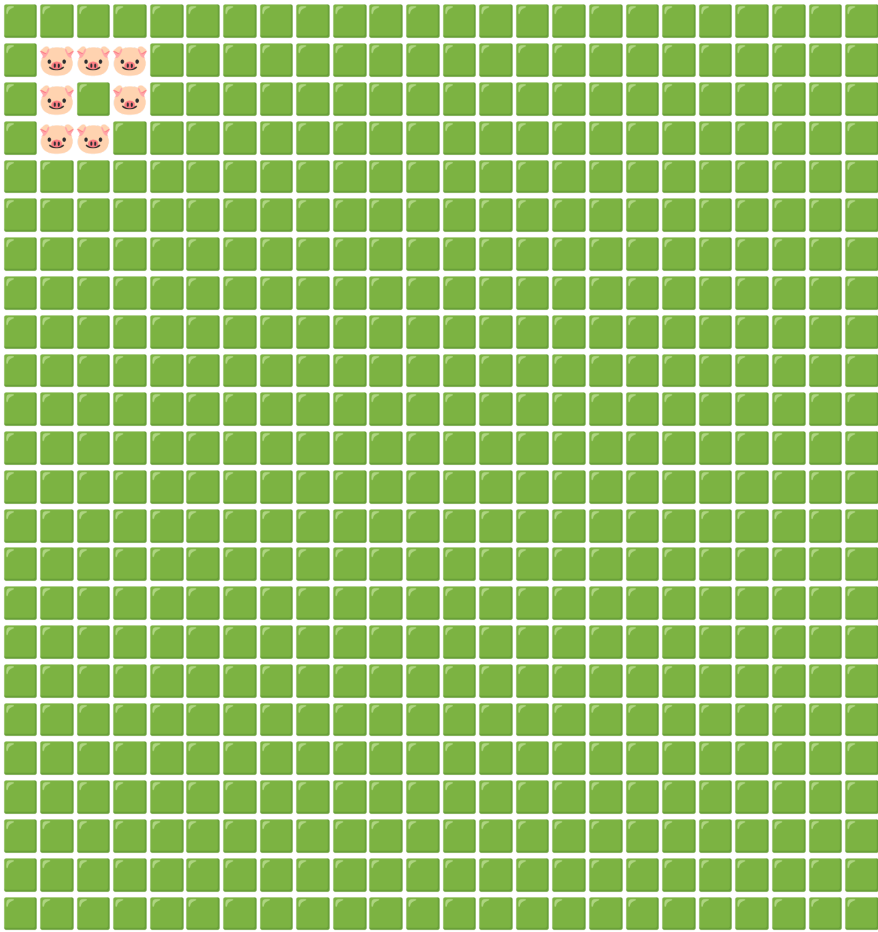
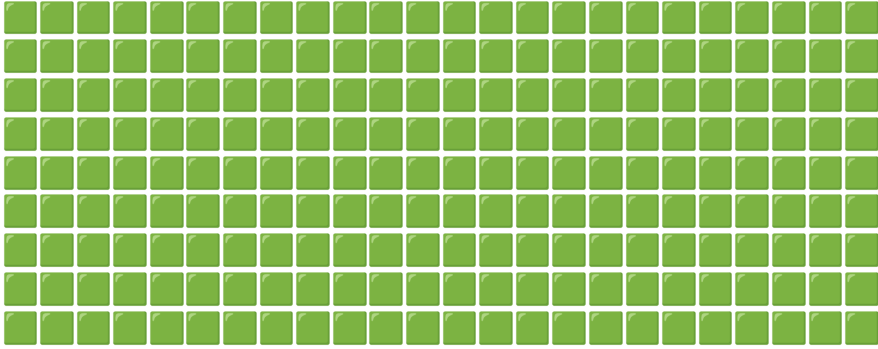
The first test checks that if there is a single point, the point dies. The second test checks that the only sustainable / non-changing structure possible is a square. Both tests work.

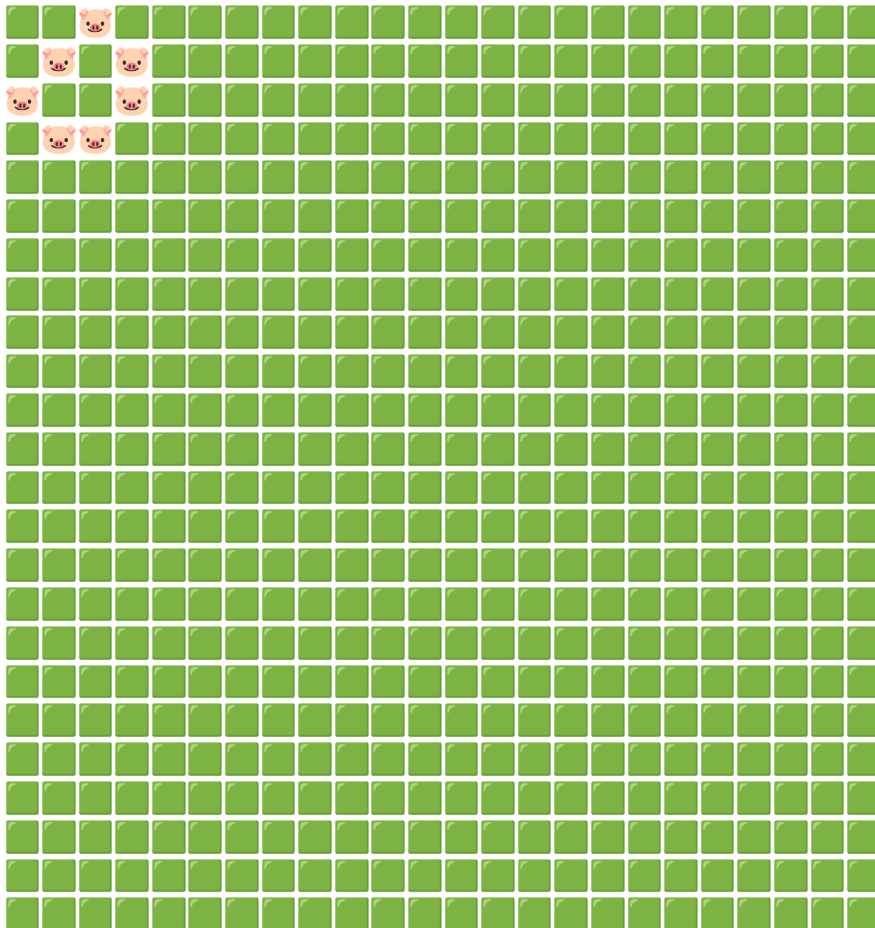
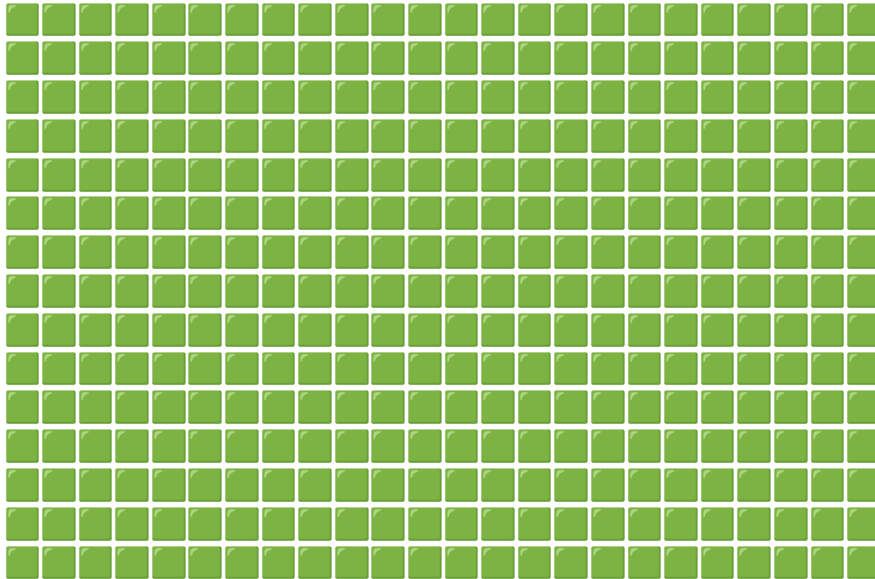
The main function creates a board with the required points initialized to alive. Then, it runs 4 iterations using a loop. In the loop, I create another empty board to initialize the next iteration. There is a nested loop inside that runs through each point and updates the new board. Outside

that nested loop, I print out the new board and set the new board to the original board to start the next iteration.

Output:







Explanation:

The given initialization ends up converging to a non-changing formation after 4 iterations. This is because each alive point has exactly 2 neighbors.